

Deadlock Avoidance, Detection and Prevention using xv6-public

Name: ROHIT MEENA Roll No: 2401MC17

1. Setup

All four programs run as ordinary xv6 user-space processes, built on top of the same kernel infrastructure added in Assignment 5 (sem_init/sem_wait/sem_post counting semaphores, shmget() for a kernel-owned shared page, sleep()/uptime()). No kernel or syscall changes were needed for this assignment -- only the five new user programs were added to UPROGS in the Makefile (see diffs/a6_Makefile.diff).

2. Banker's Algorithm (bankers.c)

Models 5 processes and 3 resource types with the standard Allocation, Max, Available and Need matrices (the classic Silberschatz textbook scenario). The Safety Algorithm finds the sequence P1, P3, P4, P0, P2 for the initial state. The Resource-Request Algorithm was tested with two requests: P1 requesting (1,0,2) passes the Need/Available bounds checks and, after a trial allocation, is confirmed safe and granted. P0 requesting (0,2,0) also passes the bounds checks but the trial safety check finds no process can finish afterwards, so it is correctly rejected with "Request denied -- would lead to unsafe state" and rolled back.

3. Deadlock Detection via Wait-For Graph (deadlockdetect.c)

Builds a wait-for graph from hardcoded allocation/request matrices (edge $P_i \rightarrow P_j$ when P_i waits on a resource held by P_j) and runs a DFS-based cycle detector. Scenario 1 (P1 waits on P0, P3 waits on P1) is acyclic and correctly reports no deadlock. Scenario 2 introduces a 3-process circular wait $P0 \rightarrow P1 \rightarrow P2 \rightarrow P0$, plus an extra process P3 blocked on the cycle without being part of it; the program correctly detects and prints the exact cycle $P0 \rightarrow P1 \rightarrow P2 \rightarrow P0$.

4. Deadlock Prevention via Resource Ordering (ro_bad.c / ro_fixed.c)

Two real xv6 processes (forked, not simulated) contend for two semaphore-based locks. In ro_bad.c, Process A acquires Lock1 then Lock2 while Process B acquires Lock2 then Lock1; each process sleeps for several ticks between its two acquisitions to force a genuine context switch in the race window (a plain busy-wait loop finished within a single scheduler quantum and did not reproduce the race). The captured log (logs/q3_resourceorder_bad_HANGS.log) shows A holding Lock1 and waiting on Lock2 while B holds Lock2 and waits on Lock1 -- a genuine circular wait -- after which the shell prompt never returns and the run had to be terminated from the host. In ro_fixed.c, both processes acquire the locks in the same global order (Lock1 before Lock2); with the identical timing, B now blocks on Lock1 until A releases it, and both processes complete and print their "done" message. Consistent lock ordering removes the possibility of a cycle in the resource-allocation graph, which is precisely the circular-wait condition needed for deadlock, so eliminating it eliminates deadlock by construction.

5. Combined Synchronization and Avoidance (syncdeadlock.c)

Five forked processes each need 2 of 3 resource pools (2 printers, 1 scanner, 2 disks), modelled as counting semaphores. Strategy used: resource ordering, reused from Q3 -- every process always requests its lower-numbered resource type before its higher-numbered one (printer=0 < scanner=1 < disk=2), so no two processes can ever form a cycle while waiting on each other's held resources. Each process prints when it requests, is granted, starts work, and releases, across 4 cycles. A shared page (shmget()) tracks per-process completion counts and flags any resource pool that is ever over-subscribed. All 5 processes complete all 4 cycles, resources never exceed their instance counts, and no process is ever permanently blocked.

6. Results

BANKERS PASS; DEADLOCKDETECT PASS; RESOURCEORDER_BAD hangs as expected (genuine deadlock, verified in logs); RESOURCEORDER_FIXED PASS; SYNCDEADLOCK PASS. The diffs, tests, and logs provide reproducible evidence.

Question	Result
Q1 Banker's Algorithm	Safe sequence found; 1 request granted, 1 correctly denied
Q2 Deadlock Detection (RAG)	Acyclic case clean; 3-process cycle detected exactly
Q3 Resource Ordering	Bad version hangs (real deadlock); fixed version completes
Q4 Combined Sync + Avoidance	All 5 processes complete 4 cycles; no overrun, no deadlock